

FIAP · MBA AI ENGINEERING & MULTI-AGENTS
CLOUD & COGNITIVE ENVIRONMENTS

Serviços Cognitivos & APIs

Aula 04 — O agente ganha sentidos: escuta (Speech), lê com inteligência (Language) e vê (Vision). Uso de APIs cognitivas **prontas** plugadas em endpoint serverless

Como vamos passar as próximas 4 horas

00:00 — 00:55

Bloco 1 — Ecossistema cognitivo

Pronto vs custom vs LLM, pricing, Azure OpenAI em 5min + lab: provisionar AI Services multi-service

00:55 — 02:00

Bloco 2 — Speech & Segurança

STT/TTS, a mesma vacina da Aula 3 (API key vs Managed Identity) + lab: transcrever áudio

02:00 — 02:10

Break

10 minutos de pausa

02:10 — 03:10

Bloco 3 — Language

Sentimento, entidades, PII/LGPD + lab: pipeline de reviews (DB Documentos) enriquecidas

03:10 — 04:00

Bloco 4 — Vision & Matriz de decisão

Tags, OCR, object detection, pronto vs custom + lab Vision, destroy e o pipeline cognitivo

Teu agente já busca produtos. Hoje ele ganha **sentidos**.

Na Aula 3 a interação ganhou vida — a primeira **tool** do agente. Hoje plugamos mais três funcionalidades: **escutar** (Speech), **ler com inteligência** (Language), **ver** (Vision). E em cada uma vocês devem refletir: API pronta, modelo custom ou LLM?

Onde vão viver os dados dos agentes ?

Storage e bancos são decisões arquiteturais que definem o que o agente consegue ou não fazer.



01



Funções cognitivas genéricas

API pronta (Plug and Play) vs modelo customizado vs LLM — prós e contras e cobrança.
E o primeiro recurso multi-service provisionado via Terraform.

Três jeitos de colocar IA no agente

API PRONTA

Modelo genérico já treinado

Provedor treinou, você consome via REST.

Caso genérico: sentimento, OCR, transcrição.

Setup em **minutos**.

● Speech · Language · Vision

MODELO CUSTOMIZADO

Você fornece os dados

Precisa treinar/fine-tunar no seu domínio.

Vocabulário próprio: peças industriais, contratos jurídicos.

● Custom Vision · CLU

LLM (FOUNDATION)

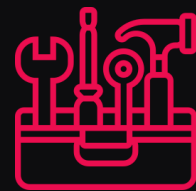
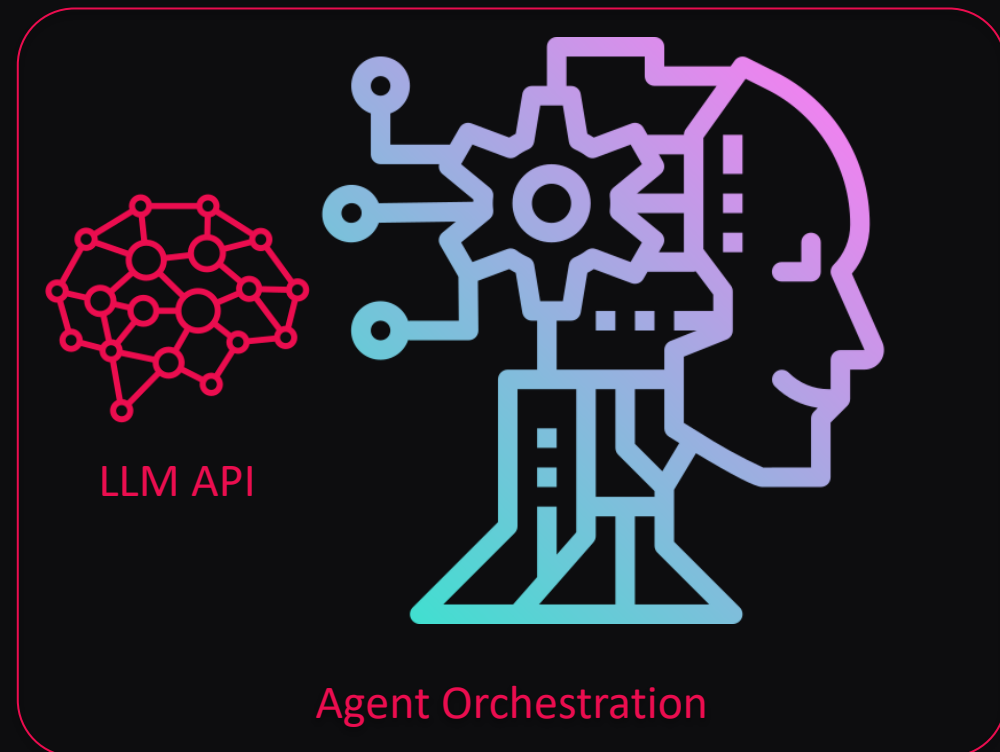
Tarefas abertas

Resumir, gerar, raciocinar.

Flexível e poderoso — mas o mais caro e o mais lento, precisa decidir o que fazer, pode resultar algo bem diverso.

● Azure OpenAI

Regra mental: tarefa genérica e fechada → pronta
domínio próprio → custom
aberta e criativa → LLM.



· A ANATOMIA ·

Um cérebro com sentidos plugados.

O agente não é um cérebro isolado. Ouvido: Speech. Visão: Vision. Compreensão: Language.

Coordenando tudo: o LLM.

Cada sentido é uma tool que você liga ou desliga — e o LLM é o maestro.

Bloco 1 · pausa visual

Tools: O melhor depende do caso

CRITÉRIO	API PRONTA	CUSTOM	LLM
Setup	minutos	dias—semanas	minutos
Custo unitário	\$/chamada (baixo)	\$/chamada (médio)	\$/token (alto)
Latência	<500ms	<500ms	1–5s
Controle	baixo	alto	médio
Domínio genérico	4 / 5	3 / 5	4 / 5
Domínio específico	2 / 5	5 / 5	3 / 5

Protótipos e MVPs,
depois migra para APIs
prontas ou modelos
custom

API PRONTA

por uso

por chamada, caractere ou Segundo

Vision ~\$1 / 1k img

Language ~\$2 / 1M chars

Speech ~\$1 / hora

CUSTOM

uso + treino

custo de treinamento + chamada

Variável pelo problema, necessita equipe especializada, CAPEX alto, OPEX baixo

Obs.: Custom Vision treino ~\$1–2 / h

+ storage do modelo

+ predição por chamada

LLM

por token

input + output, varia por modelo

Azure OpenAI ~\$2–15 / 1M token

10–50× mais caro em escala

Em escala, trocar uma API pronta por LLM pode multiplicar a conta por 10–50× .

Se a tarefa cabe num serviço pronto, use o pronto.

Guardem isso pro exercício de custo.



Open AI - O LLM gerenciado da Azure

API gerenciada para os modelos da **OpenAI** (GPT-4o, embeddings) e da própria Microsoft (Phi, DeepSeek), rodando dentro do Azure.

PARA A DISCIPLINA – CONTEÚDO DA AULA

Gerar descrição de produto, resumir reviews, embeddings para RAG.

ONDE APROFUNDAR

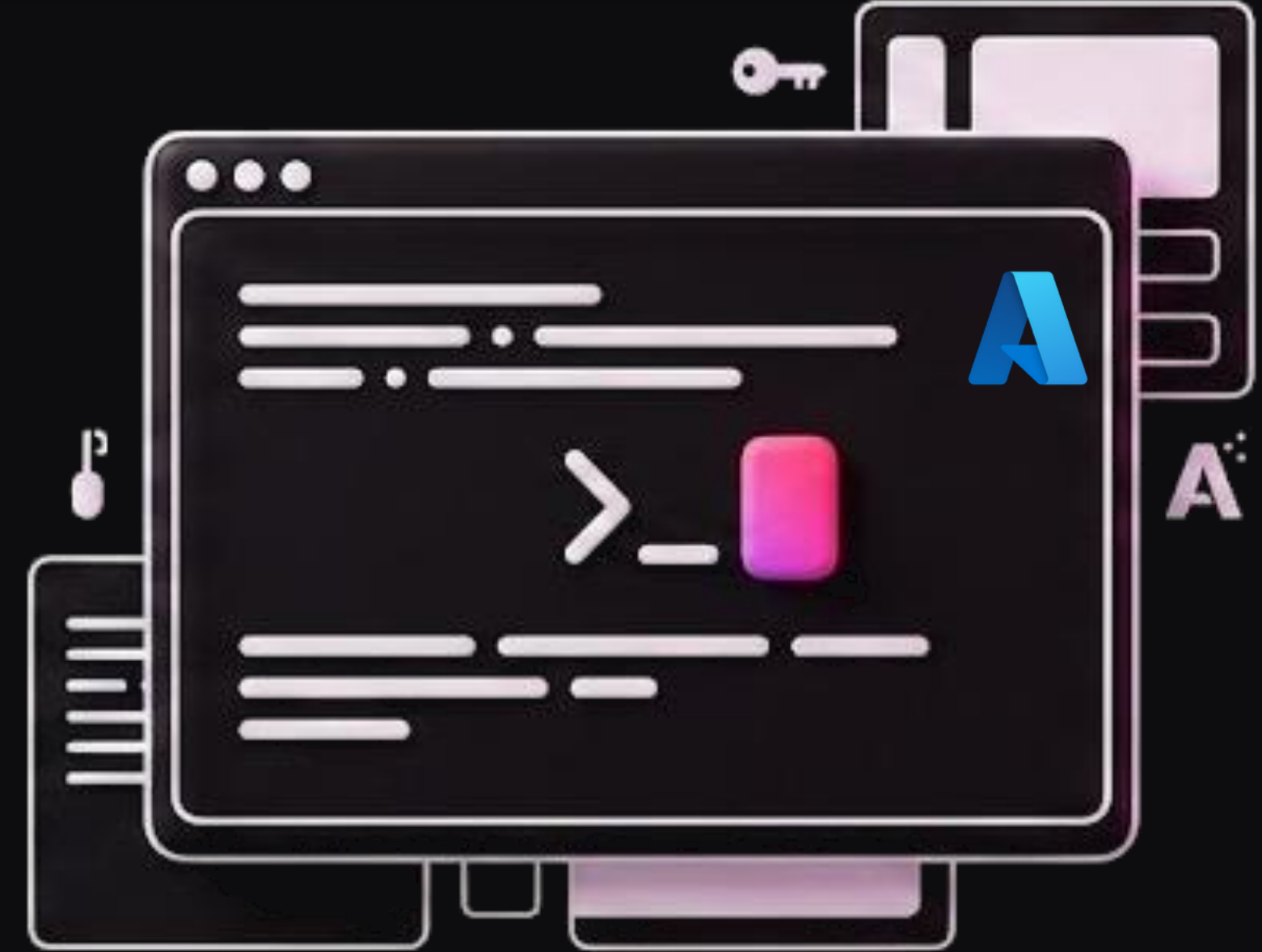
Knowledge Management & Prompt Engineering — outra disciplina do MBA.

QUER IR ALÉM AINDA HOJE?

Exercício N3 gera **embeddings reais** e fecha o loop da busca vetorial da Aula 2.

vs OpenAI direta: dados não vão pra treino , hosting no Azure (LGPD), Managed Identity e isolamento de rede.

LAB 1



Provisionamento + Mongo DB - Terraform

Antes de continuar, certifique-se de ter à disposição:

- Conta Azure já logada
- Cloud Shell funcionando
- Uma boa xícara de café ☕

1 – Preparação – cria os componentes requeridos antes do lab

Clonar o repositório (apenas na primeira vez)

```
git clone https://github.com/isaiasbritto/aie-cloud.git  
cd aie-cloud/aulas/04-servicos-cognitivos/lab/terraform
```

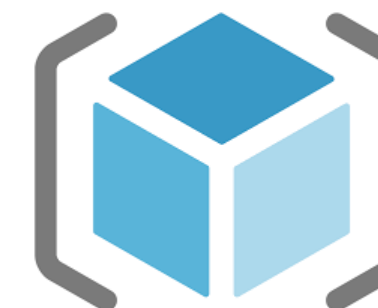
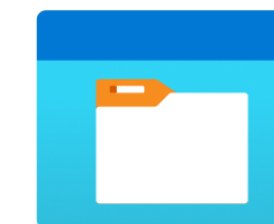
Abrir o editor VS Code na pasta do lab
code .

Inicializar providers (download de bibliotecas)
terraform init

Ver o que será criado
terraform plan

Aplicar
terraform apply -auto-approve

Caso o repo tenha sido clonado, apague antes com
rm -rf aie-cloud



Mongo DB
Novo integrante, no
lugar do Cosmos DB

Aproximadamente ~5 a ~8 minutos (tempo para explorar a pasta terraform)

Lembrete da regra de ouro:

Ao final, devemos encerrar tudo com **Exclusão do RG** ou com `terraform destroy`

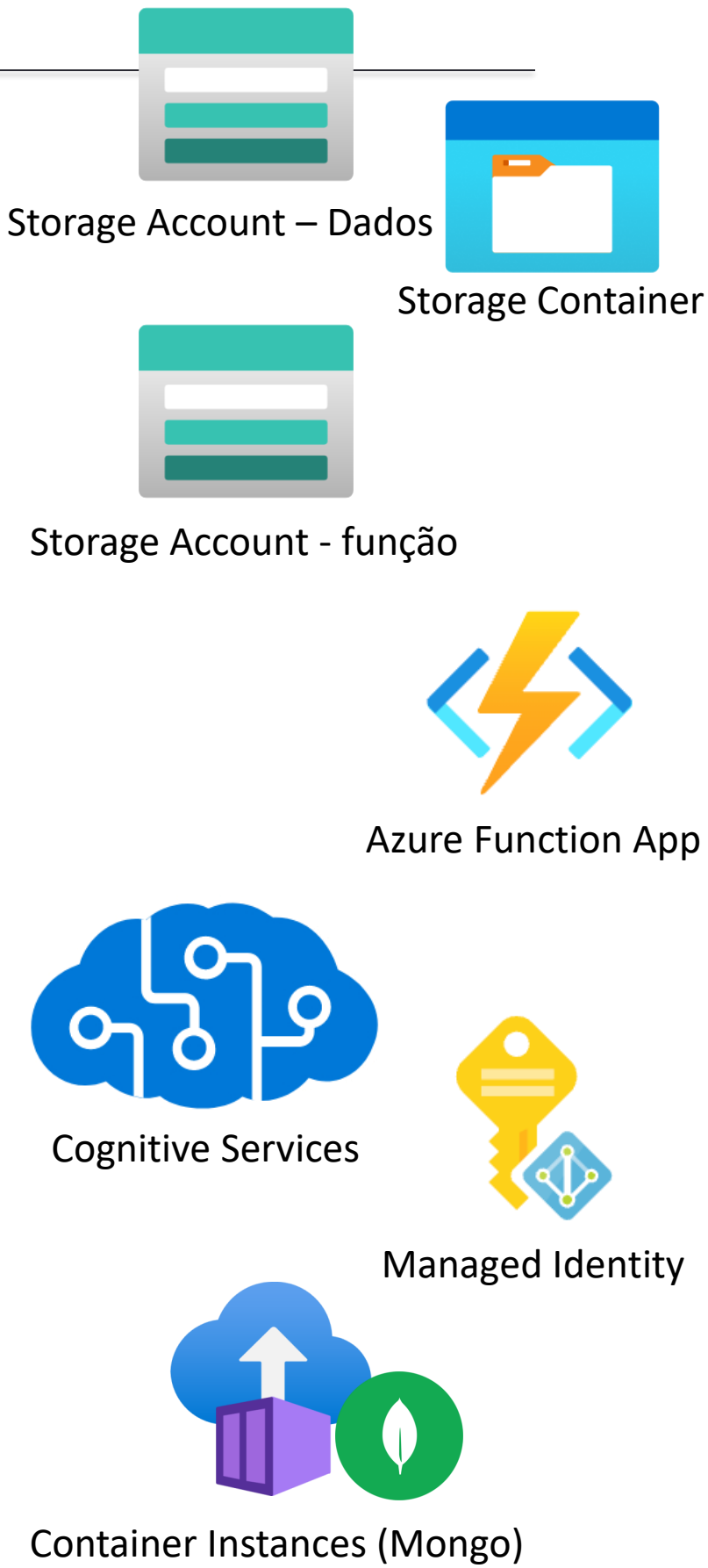
2 – Enquanto o terraform trabalha, vamos explorar o que temos nos arquivos “.tf” da aula [aie-cloud/aulas/04-servicos-cognitivos/lab/terraform](#)

Arquivo	O que define
main.tf	Providers, RG, locals (tags + credenciais Mongo), data source de identidade
variables.tf	location (default: eastus2)
outputs.tf	Todos os outputs consumidos pelo guia e pelos scripts
storage.tf	Storage Account de dados + containers audios e imagens
function.tf	Storage da Function + App Service Plan + Function App + role assignment
cognitive.tf	Azure AI Services multi-service (Speech, Language, Vision)
keyvault.tf	Key Vault + role para o usuário + segredo ai-services-key
mongodb.tf	Container Group ACI: MongoDB 7.0 + Mongo Express 1.0.2

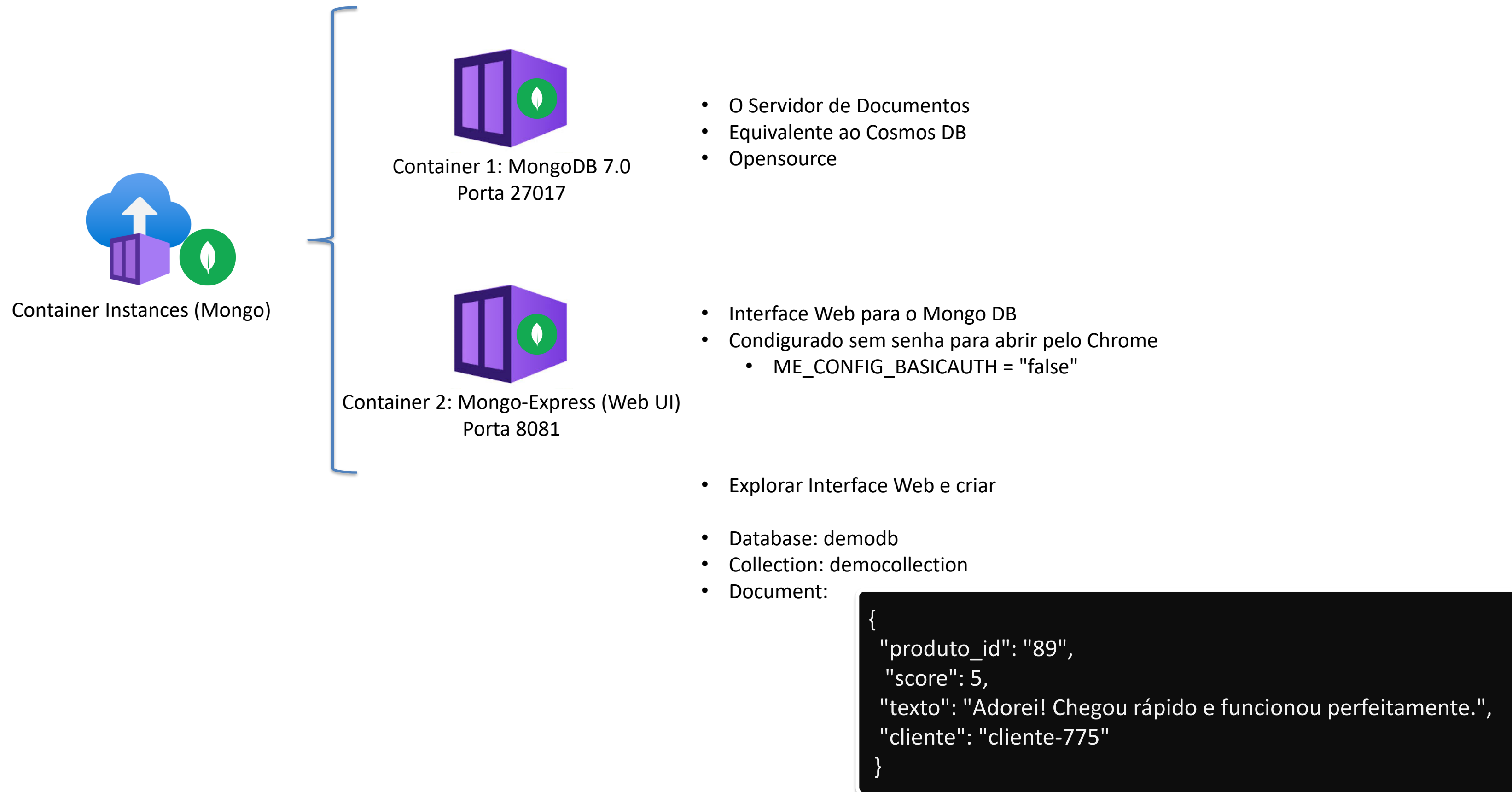
```
# Consultar recursos criados pelo terraform
terraform output
```



Resource Group



► LAB 1 – Terraform da aula 4



Avaliação do arquivo Python

- Localize o diretório scripts usando o VS Code;
- Explore o arquivo semear_reviews.py, veja seu conteúdo;
- Execute o Script Python:

Instalar dependências do Script

```
pip install --user pymongo -q
```

Obtem o IP do Mongo DB

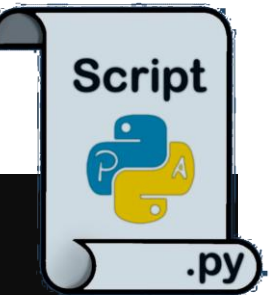
```
export MONGO_IP=$(cd ~/aie-cloud/aulas/04-servicos-cognitivos/lab/terraform &&  
terraform output -raw mongodb_public_ip)  
echo "IP do Mongo: $MONGO_IP"
```

Execução do script (confira o diretório atual)

```
python3 scripts/semear_reviews.py
```

Resumo de semear_reviews.py

```
import os, sys  
from pymongo import MongoClient, errors  
  
app = func.FunctionApp(http_auth_level=func.AuthLevel.ANONYMOUS)  
  
URI = f"mongodb://{admin:QCadmin2024!@{MONGO_IP}:27017/?authSource=admin"  
  
client = MongoClient(URI, serverSelectionTimeoutMS=5000)  
client.server_info()  
print("✓ MongoDB pronto!")  
  
db = client["qc-db"]  
reviews_col = db["reviews"]  
  
reviews = [  
    {"id": "r001", "produto": "Notebook Pro X", "texto": "Excelente laptop, a bateria dura mais de 8 horas e o  
    processador é muito rápido.", "estrelas": 5},  
    {"id": "r002", "produto": "Fone Bluetooth QC-50", "texto": "Qualidade de som péssima, muito abaixo do  
    esperado. Totalmente decepcionante.", "estrelas": 1},  
]  
  
result = reviews_col.insert_many(reviews)  
  
client.close()
```



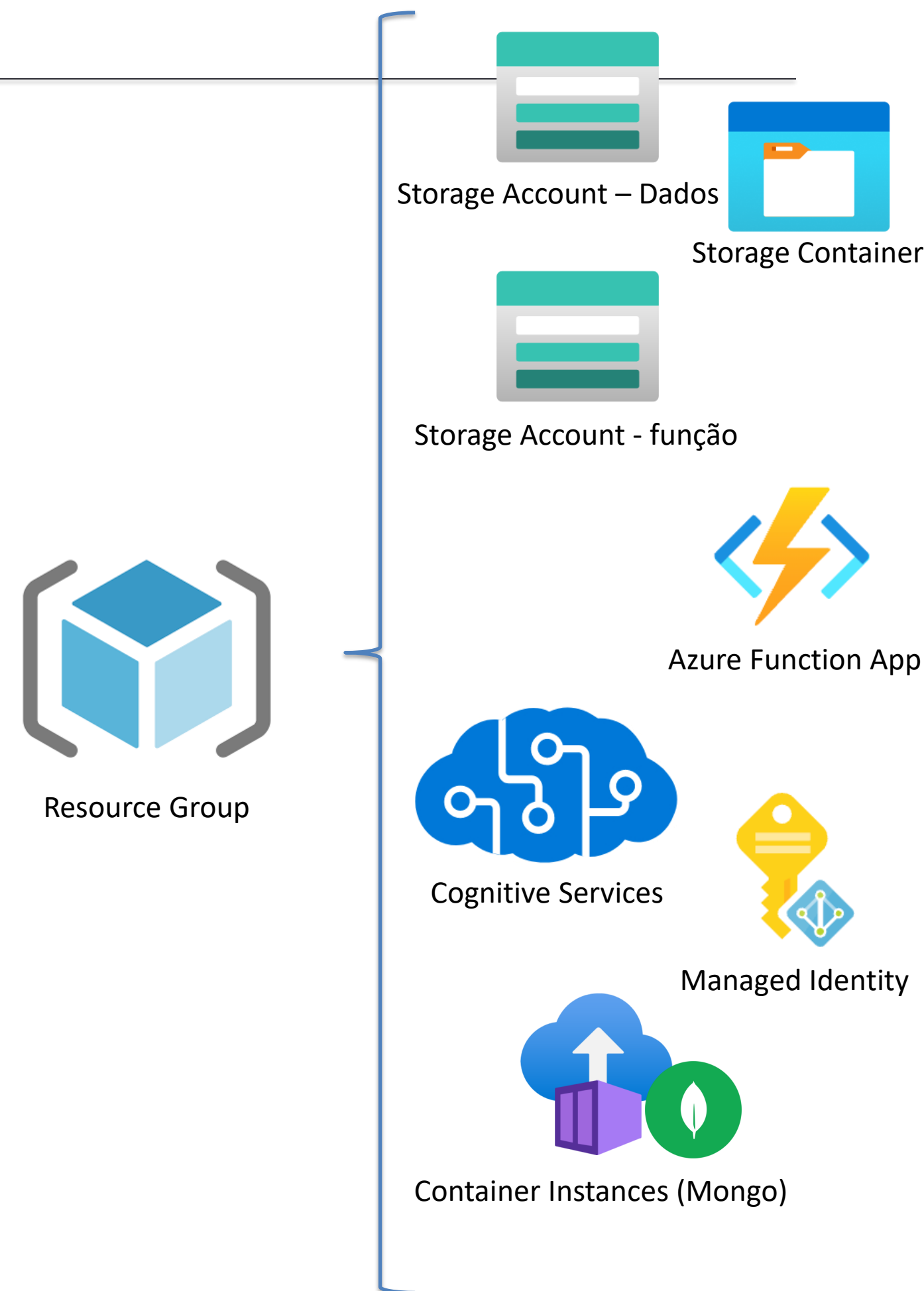
Você identificou alguma falha de segurança?

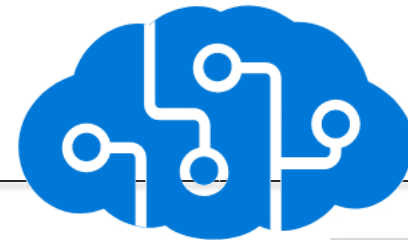
Exercício: Como consertar?

Ao final, avalie o conteúdo pela interface interativa (Web). Navegue até qc-db → reviews para ver os documentos.

No portal Azure → Resource Group rg-qc-aula04-*:

- 2 × Storage Account (Function runtime + dados do lab)
- 1 × App Service Plan (Y1 Consumption)
- 1 × **Cognitive Services** (multi-service S0 com custom subdomain)
- 1 × **Key Vault** (com segredo ai-services-key)
- 1 × **Azure Container Instance ACI** (MongoDB 7.0 + Mongo Express)
- 1 × **Function App** (Python 3.11, Managed Identity SystemAssigned)



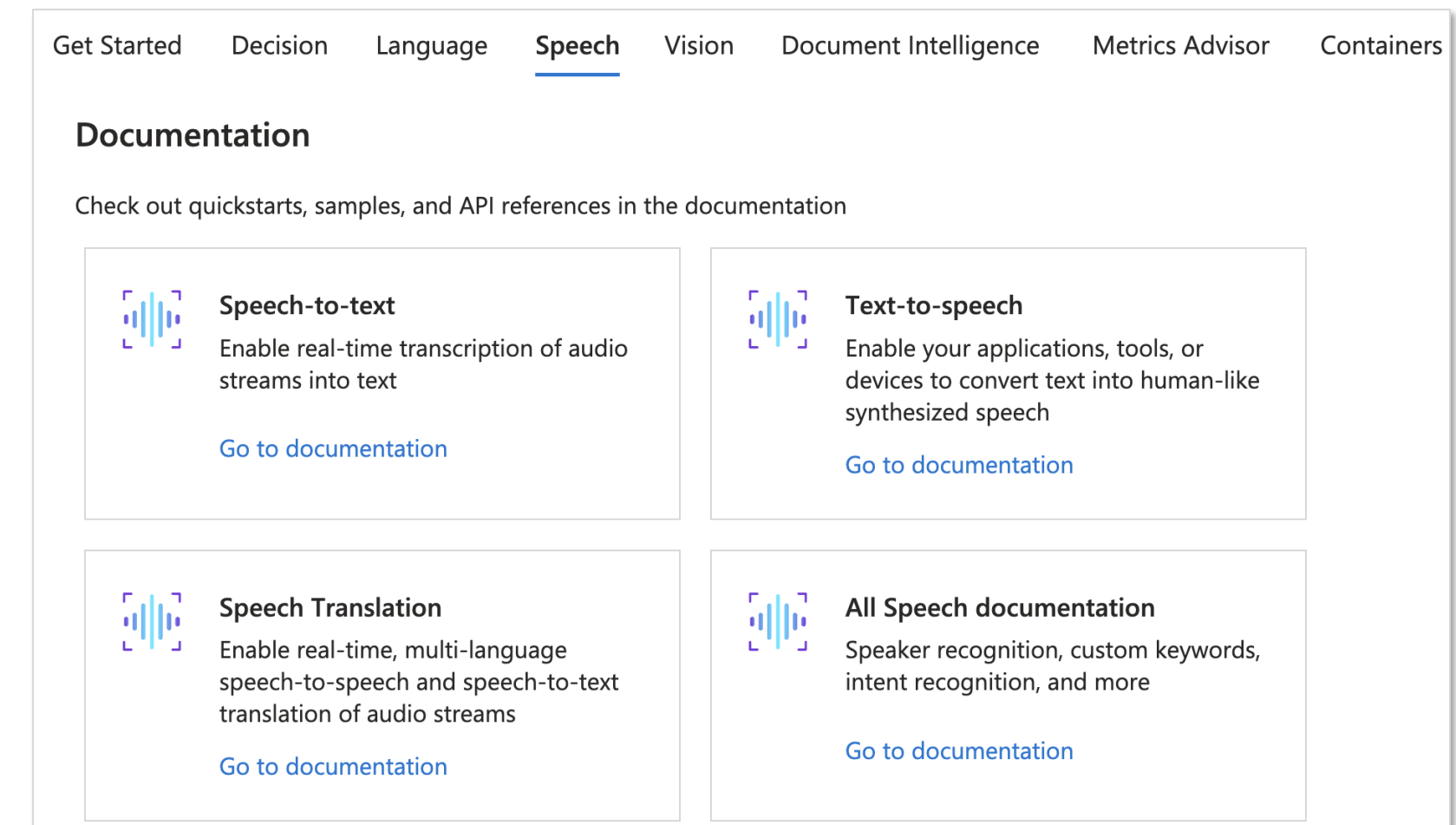
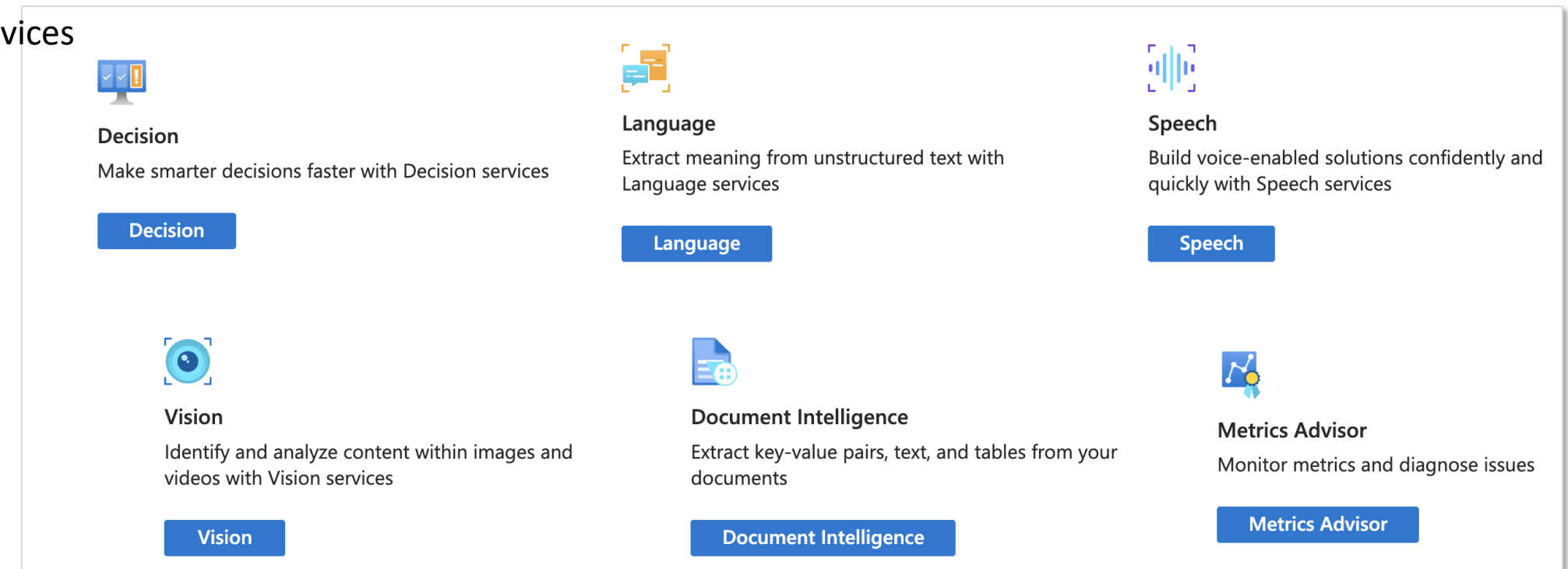


Cognitive Services

– Localize no rg criado um serviço APIs cognitivas.

Avalie:

- Overview
 - Subprodutos
- Keys and Endpoint
 - Chave 1 e Chave 2
- Access control (IAM)
 - Managed Identity
- Discussão: Existem muitos serviços similares na internet, conheço muitas pessoas que vivem de SaaS como esses.



02



Speech & Segurança

Com Serviços prontos, o agente aprende a escutar.

E a mesma vacina da Aula 3 — Managed Identity em vez de chave no código — aplicada agora a serviços cognitivos.

Os dois lados da voz

SPEECH-TO-TEXT

Áudio → texto

Real-time (websocket) p/ atendimento ao vivo (Case Sky)

Batch (URL → resultado)

Diarização (separar falantes), pontuação automática, profanity filter.

Custom Speech treina vocabulário próprio.

- Um dos recursos genéricos mais usados

TEXT-TO-SPEECH

Texto → voz

Vozes neurais PT-BR muito naturais: **Antonio, Francisca, Brenda** .

SSML controla entonação, pausa, ênfase.

Casos: URA, audiobooks, acessibilidade.

No lab serve de fallback p/ gerar áudio de teste.

- Vamos fazer uma geração de áudio

· A MESMA VACINA ·

Agente não usa API key chumbada no código.

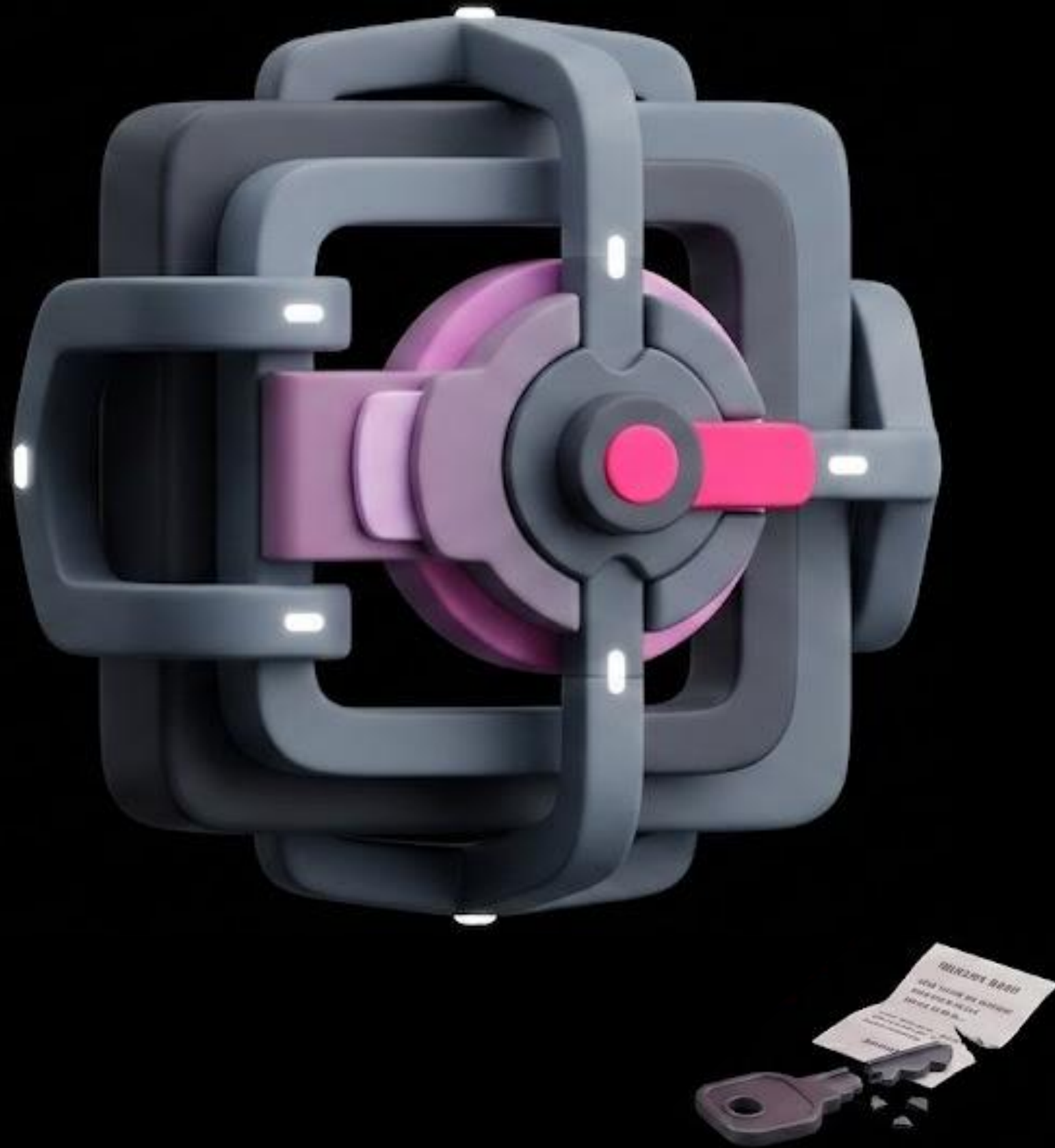
É a vacina da Aula 3, recurso diferente.

A chave vaza no primeiro push, igual à senha.

Managed Identity de novo: a Function tem identidade, ganha a role, e o token vem sozinho.

Mesma técnica, novo serviço.

Bloco 2 · pausa visual



Da mais simples à mais segura

MÉTODO	COMO	QUANDO
API Key	Ocp-Apim-Subscription-Key: <key>	Dev/test rápido · legado
Token AAD · Managed Identity	Authorization: Bearer <token>	Padrão em produção
Custom Domain + Network	Restrição IP/VNet + auth	Compliance enterprise

#1 API Key

```
client = TextAnalyticsClient(endpoint,
                             AzureKeyCredential(key))
```

#2 Managed Identity

```
client = TextAnalyticsClient(endpoint,
                             DefaultAzureCredential())
```

Pré-requisitos da MI: custom subdomain habilitado + role Cognitive Services User na identidade.

Da mais simples à mais segura

MÉTODO	COMO	QUANDO
API Key	Ocp-Apim-Subscription-Key: <key>	API Key: você manda a chave no header Ocp-Apim-Subscription-Key. Funciona, é rápido pra dev e test, mas ainda tem chave que pode vazar.
Token AAD · Managed Identity	Authorization: Bearer <token>	Token AAD via Managed Identity: o cabeçalho vira Authorization Bearer, e o token é negociado sozinho. É o padrão de produção.
Custom Domain + Network	Restrição IP/VNet + auth	E o terceiro nível, pra compliance enterprise: custom domain mais restrição de rede, IP ou VNet, por cima da autenticação.

#1 API Key

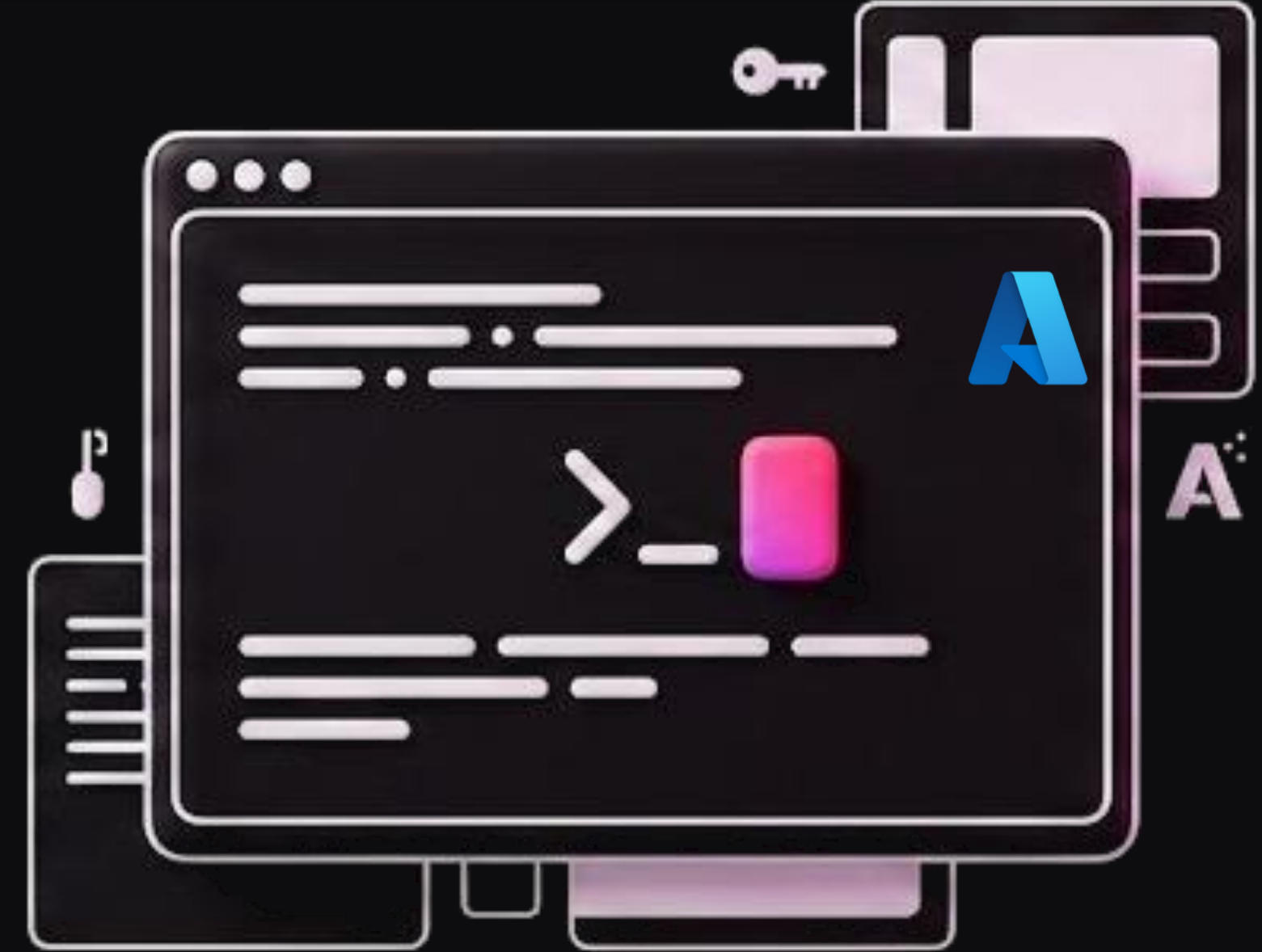
```
client = TextAnalyticsClient(endpoint,  
                             AzureKeyCredential(key))
```

#2 Managed Identity

```
client = TextAnalyticsClient(endpoint,  
                             DefaultAzureCredential())
```

Pré-requisitos da MI: custom subdomain habilitado + role Cognitive Services User na identidade.

LAB 2



Text-to-Speech e Vice Versa

Antes de continuar, certifique-se de ter à disposição:

- Conta Azure já logada
- Cloud Shell funcionando
- Uma boa xícara de café ☕

Avaliação do arquivo Python

- Localize o diretório scripts usando o VS Code;
- Explore o arquivo gerar_audio_tts.py, veja seu conteúdo;
- Execute o Script Python:

Instalar dependências do Script

```
pip install --user requests -q
```

Obtém Key Vault e Region

```
export KEY_VAULT_NAME=$(cd ~/aie-cloud/aulas/04-servicos-  
cognitivos/lab/terraform && terraform output -raw key_vault_name)  
echo "Key Vault: $KEY_VAULT_NAME"
```

Define sua Region

```
export AI_REGION="eastus2"
```

Obtém Chave do AI Services (script apenas)

```
export AI_KEY=$(az keyvault secret show \  
--vault-name "$KEY_VAULT_NAME" \  
--name ai-services-key \  
--query value -o tsv)  
echo "AI_KEY exportado (${#AI_KEY} chars)"
```

Execução do script (confira o diretório atual)

```
python3 scripts/gerar_audio_tts.py
```

Resumo de gerar_audio_tts.py

```
import os, sys
import requests

AI_REGION = os.environ.get("AI_REGION", "eastus2")
AI_KEY = os.environ.get("AI_KEY", "")

SSML = """
<speak version='1.0' xml:lang='pt-BR'>
<voice xml:lang='pt-BR' xml:gender='Male' name='pt-BR-AntonioNeural'>
A Quantum Commerce é uma plataforma de e-commerce que opera em doze países.
Nossos agentes de inteligência artificial ajudam clientes a encontrar produtos,
calcular fretes, processar pedidos e responder dúvidas em tempo real.
A nossa missão é tornar o comércio mais humano, inteligente e acessível.
</voice>
</speak>
"""

tts_url = f"https://{AI_REGION}.tts.speech.microsoft.com/cognitiveservices/v1"
# Autenticação por chave: header Ocp-Apim-Subscription-Key (não Authorization)
headers = {
    "Ocp-Apim-Subscription-Key": AI_KEY,
    "Content-Type": "application/ssml+xml",
    "X-Microsoft-OutputFormat": "riff-16khz-16bit-mono-pcm",
}

print(f"Gerando áudio via TTS ({AI_REGION})...")
resp = requests.post(tts_url, headers=headers, data=SSML.encode("utf-8"), timeout=30)

output_path = "/tmp/audio-teste.wav"
with open(output_path, "wb") as f:
    f.write(resp.content)
```



audio-teste.wav

Copiar o Áudio Gerado para a Storage Account

Obtém Data Storage

```
export DATA_STORAGE=$(cd ~/aie-cloud/aulas/04-servicos-cognitivos/lab/terraform  
&& terraform output -raw data_storage_account_name)  
echo "Data Storage: $DATA_STORAGE"
```

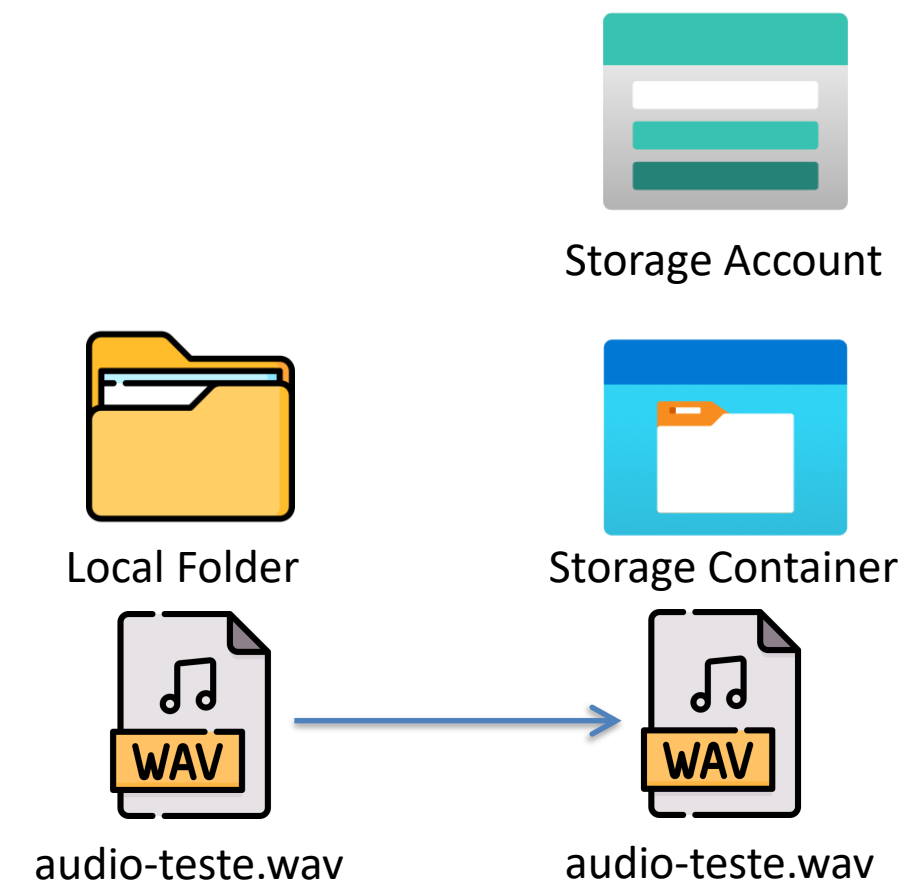
Cópia arquivo de som para Data Storage

```
az storage blob upload \  
  --account-name "$DATA_STORAGE" \  
  --container-name audios \  
  --name audio-teste.wav \  
  --file /tmp/audio-teste.wav \  
  --overwrite
```

Caso queira, substitua
por um arquivo .wav
próprio

```
az storage blob upload \  
  --account-name "$DATA_STORAGE" \  
  --container-name audios \  
  --name bbc_exemplo.mp3 \  
  --file ~/aie-cloud/aulas/04-servicos-cognitivos/lab/data/bbc_exemplo.mp3 \  
  --overwrite
```

Challenge:
O código está fixo para .wav
O que deve ser alterado para .mp3?



Deploy de Função Serverless

Obtém APP de Funções

```
export FUNC_NAME=$(cd ~/aie-cloud/aulas/04-servicos-cognitivos/lab/terraform &&  
terraform output -raw function_app_name)  
echo "Func APP Name: $FUNC_NAME"
```

Altera para o diretório das funções

```
cd ~/aie-cloud/aulas/04-servicos-cognitivos/lab/function
```

Publica Functions

```
func azure functionapp publish "$FUNC_NAME" --python
```



function

(function_app.py, requirements.txt, etc.)



Azure Function App

Pelas URLs exibidas no terminal, acesse sua função usando o browser. E pelo celular?

Quem consegue acessar por curl?

⚠ ESPERADO, NÃO É FALHA

Cold start de 2–3s

A **primeira** chamada demora — a função precisa acordar. As seguintes são milissegundos.

Como você já conhece funções Serverless, explore os códigos das rotas/funções:

- health
- transcrever (opções de parâmetros)

Adicionais:

- analisar-reviews (lab3)
- analisar-imagem (lab4, opções de parâmetros)

```
@app.route(route="transcrever", methods=["GET", "POST"])
def transcrever(req: func.HttpRequest) -> func.HttpResponse:
    """
    Transcreve áudio WAV do Blob via Azure Speech STT (REST API).
    GET /api/transcrever?blob=audio-teste.wav&idioma=pt-BR
    """

    blob_name = req.params.get("blob", "audio-teste.wav")
    container = req.params.get("container", "audios")
    idioma = req.params.get("idioma", "pt-BR")

    # 1. Baixar áudio do Blob via Managed Identity
    blob_client = _blob_service.get_blob_client(container=container, blob=blob_name)
    audio_bytes = blob_client.download_blob().readall()

    # 2. Obter speech token via key (ao invés do Key Vault)
    token = _get_speech_token(AI_REGION)
    resp = requests.post(
        f"https://{AI_REGION}.stt.speech.microsoft.com/speech/recognition/conversation/cognitiveservices/v1",
        headers={
            "Authorization": f"Bearer {token}",
            "Content-Type": "audio/wav; codecs=audio/pcm; samplerate=16000",
            "Accept": "application/json",
        },
        params={"language": idioma, "format": "detailed"},
        data=audio_bytes, timeout=30,
    )
    resp.raise_for_status()
    data = resp.json()

    transcricao = ""
    if "NBest" in data and data["NBest"]:
        transcricao = data["NBest"][0].get("Display", "")
    elif "DisplayText" in data:
        transcricao = data["DisplayText"]

    return func.HttpResponse( json.dumps({ "transcricao": transcricao,
                                           "idioma": idioma,
                                           "blob": blob_name,
                                           "status_stt": data.get("RecognitionStatus", ""),
                                           }, ensure_ascii=False), mimetype="application/json"
    )
```

Obtém áudio com segurança MI

Obtém token (outra forma)

Identifica melhor transcrição

retorno

03



Leitura de documentos

O agente lê com inteligência: sentimento, entidades, frases-chave — e detecção de dados pessoais para LGPD.

Um pipeline real sobre as reviews de dados em Document DB.

Principais capacidades, por serviço

CAPACIDADE	O QUE FAZ	CASO PRÁTICO
Sentiment Analysis	positive / neutral / negative + confiança	Reviews dos clientes
Opinion Mining	Aspecto + sentimento ("entrega" → ruim)	Reviews granulares
Entity Recognition	Pessoas, locais, orgs, datas, produtos	Produtos mencionados
Key Phrase	Extrai Frases-chave / Palavras-chave	Dashboards · roteamento
Language Detection	identifica o idioma, pra roteamento multilíngue	Dashboards · roteamento
PII Detection	Identifica/mascara CPF, e-mail, telefone	Compliance LGPD
Summarization	Resume textos	Reviews longas
Custom CLU	NLU customizado	Identificar "intenções" para agentes

Pra aula de hoje: sentimento mais entidades nas reviews.

PII detection tem sido muito utilizado em soluções atuais que envolvem segurança.

Por que pronto e não LLM?

SENTIMENT VIA SERVICE ESPECÍFICO

~\$0,75 / 1M chars · latência <500ms · otimizado pra tarefa.

SENTIMENT VIA LLM

~\$2–15 / 1M tok · latência 1–3s · flexível, mas caro.

Cabe num serviço pronto?

Use o pronto — 10–50× mais barato em escala.



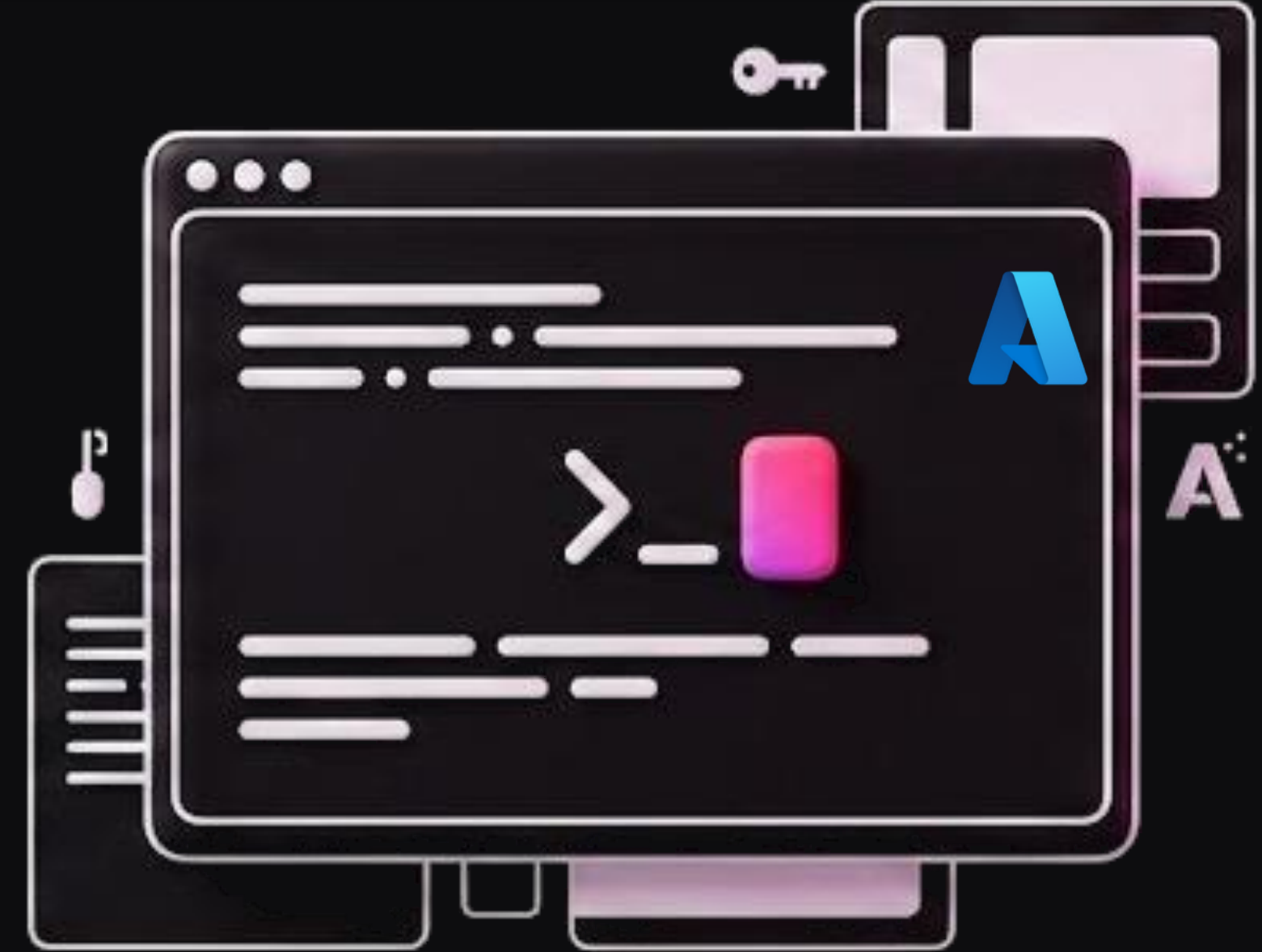
PII DETECTION · RELEVÂNCIA LGPD

Mascarar antes de logar

Reviews às vezes trazem **CPF, e-mail, telefone** que o cliente esqueceu.

O PII identifica e redige/mascara antes de logar ou treinar:
compliance automático.

LAB 3



Pipeline de reviews QC → Doc DB

Antes de continuar, certifique-se de ter à disposição:

- Conta Azure já logada
- Cloud Shell funcionando
- Uma boa xícara de café ☕

Avaliação do arquivo Python

- Localize o diretório scripts usando o VS Code;
- Explore o arquivo function_app.py e a função analisar_reviews;

- Execute o Script Shell:

```
# Obtém Hostname da Function APP
export HOSTNAME=$(cd ~/aie-cloud/aulas/04-servicos-cognitivos/lab/terraform &&
terraform output -raw function_app_hostname)
echo "Host Name: $HOSTNAME"

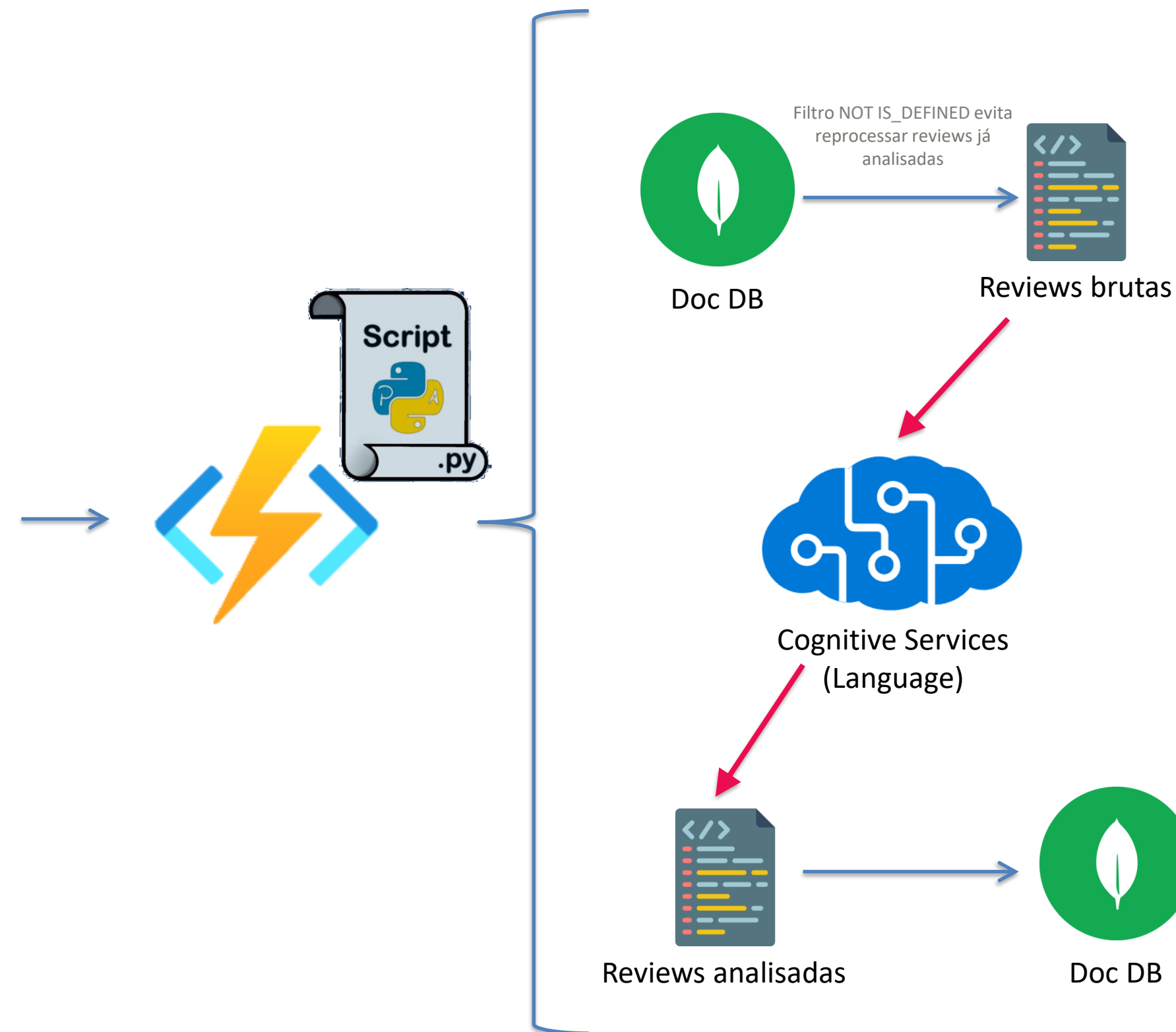
# Exibe URL de Microbatch
echo "URL para avaliar reviews: $HOSTNAME/api/analisar-reviews?limit=10"
```

- Ao final, abra o Mongo Express (porta 8081) para consultar as reviews no mongo DB.

Database qc-db

Collection: revies

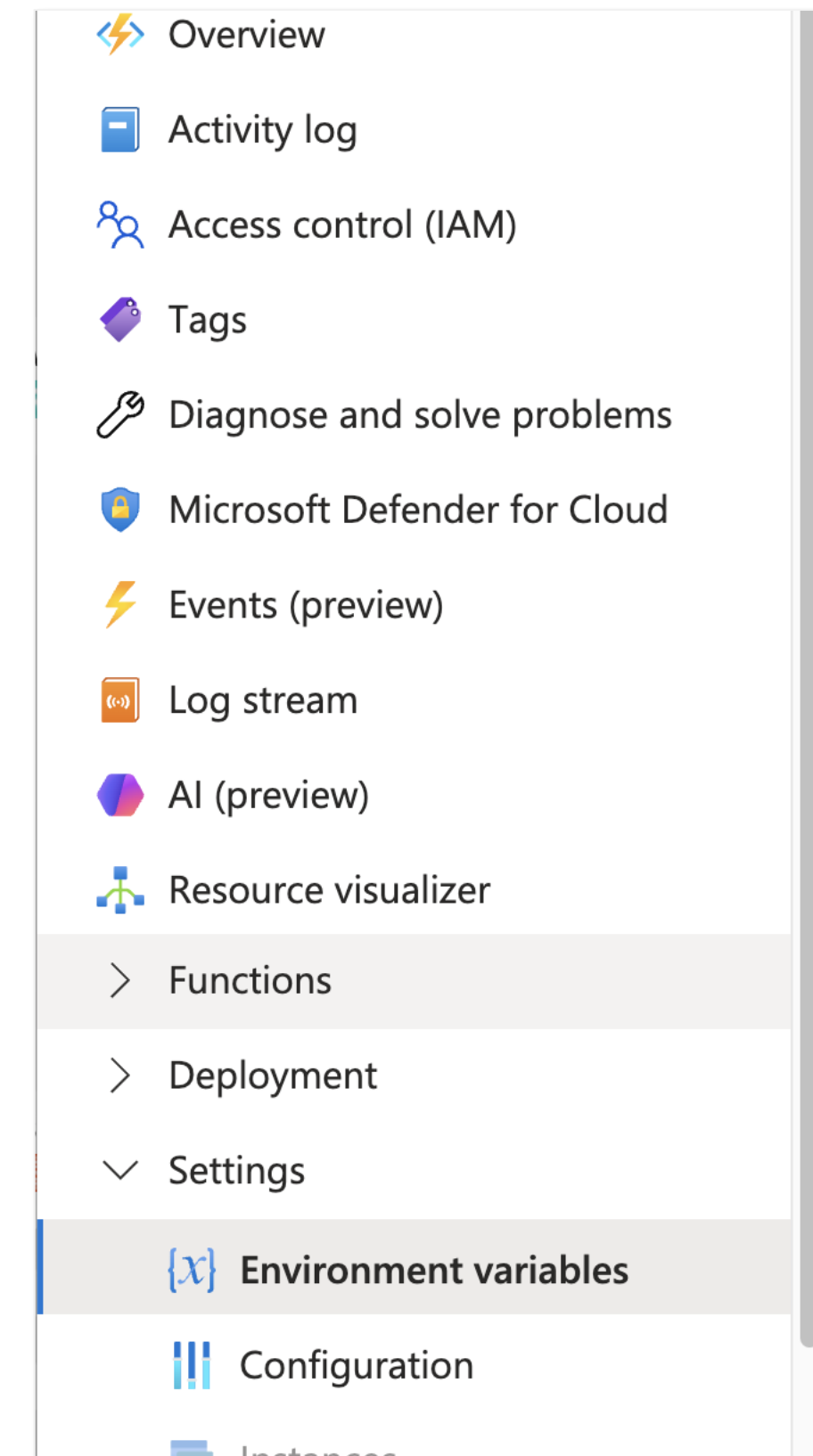
```
# Caso tenha fechado a tela do Mongo Express:
echo "Mongo Express: http://$(cd ~/aie-cloud/aulas/04-servicos-
cognitivos/lab/terraform && terraform output -raw mongodb_public_ip):8081 "
```



Faça uma nova análise do pipeline, agora focado em segurança

Observe:

- Dados do Mongo DB por variável de ambiente
- AI KEY por Key Vault
- Nenhum valor exibido ou exposto
- Nenhum valor salvo no git (ao menos, não na função)



04



Vision & Matriz de decisão

O agente ganha visão — tags, OCR, object detection

Entrega conceitual da aula: a matriz final pronto vs custom vs LLM.

Destroy e o pipeline cognitivo completo.

Principais serviços de Vision AI

IMAGE TAGGING

Lista tags

"furniture", "chair", "office" → auto-categorizar produto.

OBJECT DETECTION

Bounding boxes

Localiza itens na foto → contar produtos na prateleira.

OCR · READ API

Extraí texto

Etiqueta, embalagem, documento → texto estruturado.

IMAGE DESCRIPTION

Caption natural

Descrição em linguagem natural → acessibilidade.

IMAGE EMBEDDING

Vetor (Florence)

Busca visual → "encontre fotos similares".

SMART CROPPING

Recorte focado

Thumbnail centrado no produto.

"É uma cadeira?" vs "é qual modelo nosso?"

VISION PRONTO

Vocabulário genérico

Classifica em chair, table, food, person... Resolve **~80%** dos casos sem treinar nada.

- setup em minutos

CUSTOM VISION

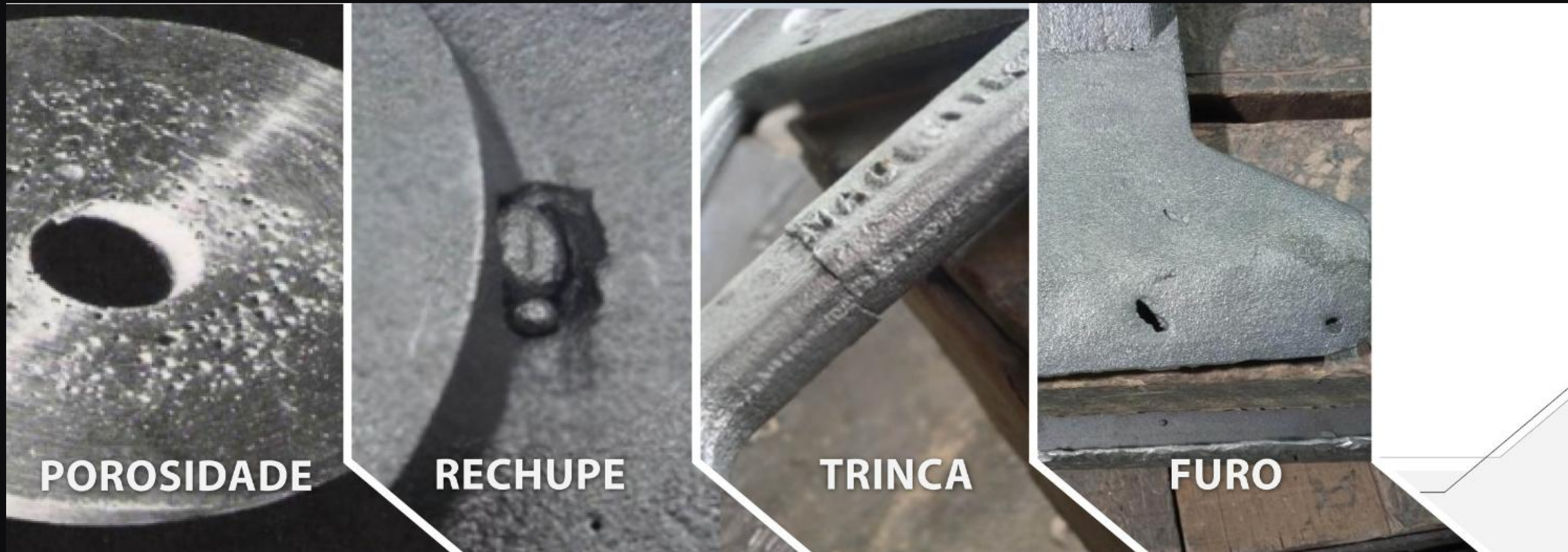
Vocabulário próprio

Você define as classes (sofá-3-lugares, poltrona...), **30–50 imagens** por classe, treino de 10–30min por ~\$1–2.

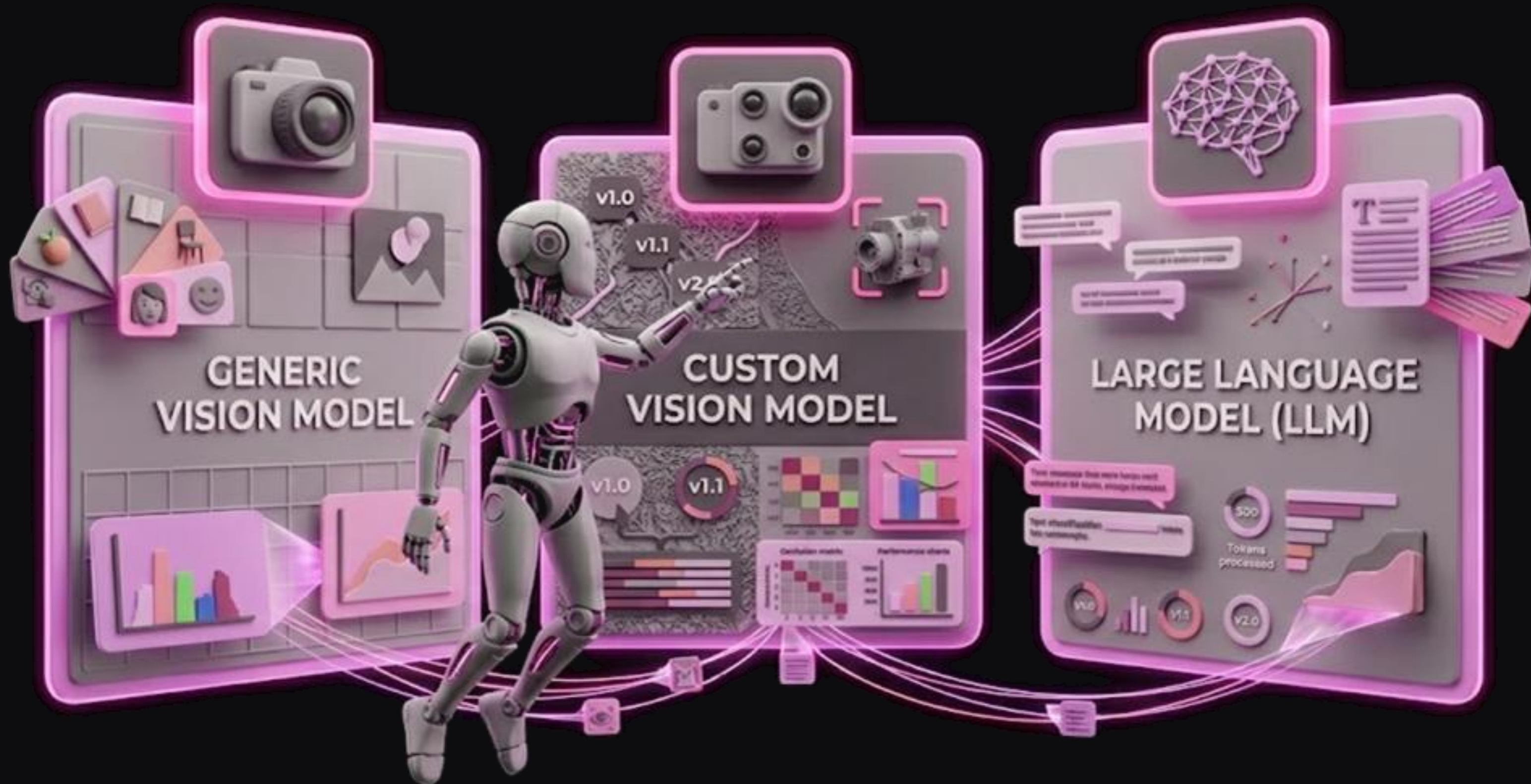
- linha Premium

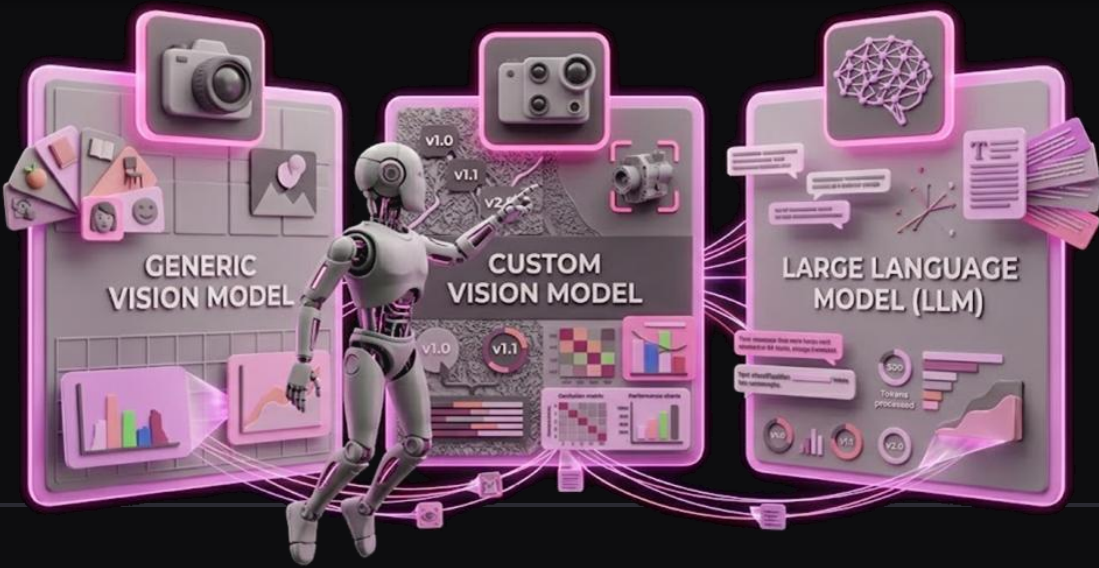
A regra: vocabulário genérico → pronto
 vocabulário próprio → custom.

Caso claro de Custom Model: Defeito em peças de linha de produção



E uso de LLM para visão?



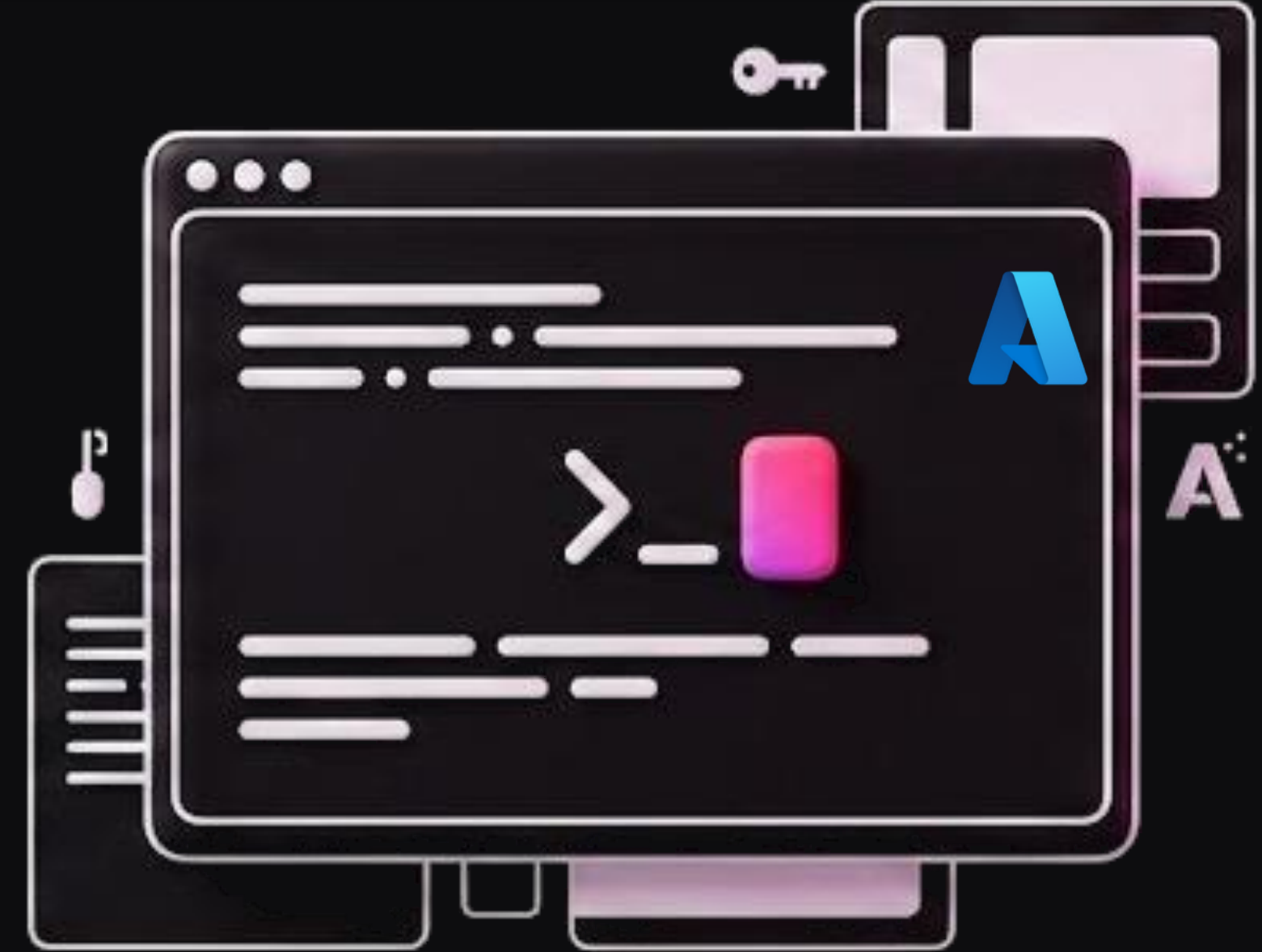


Uma matriz de decisão conceitual

CENÁRIO	RECOMENDADO
"É uma cadeira?" — vocabulário genérico	Vision (pronto)
"É qual modelo da nossa marca?" — vocabulário próprio	Custom Vision
"Resuma esta etiqueta nutricional"	OCR + LLM
"Encontre produtos similares à foto"	Image Embeddings + AI Search
"Detecte produtos numa foto de prateleira"	Object Detection (custom)

Reparem no padrão: as melhores soluções combinam serviços.
OCR extrai, LLM interpreta. Embedding vetoriza, Search busca.
O agente é o maestro.

LAB 4



Vision: classificação + OCR

Antes de continuar, certifique-se de ter à disposição:

- Conta Azure já logada
- Cloud Shell funcionando
- Uma boa xícara de café ☕

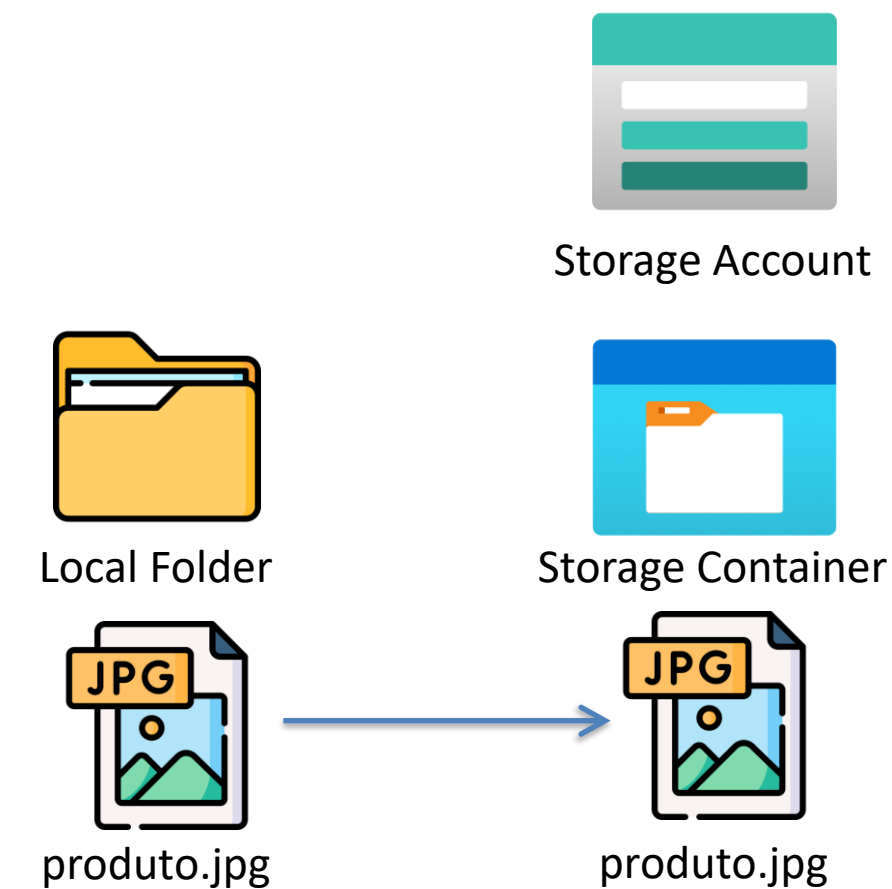
Copiar imagem para a Storage Account

Obtém Data Storage

```
export DATA_STORAGE=$(cd ~/aie-cloud/aulas/04-servicos-cognitivos/lab/terraform && terraform output -raw data_storage_account_name)
echo "Data Storage: $DATA_STORAGE"
```

Cópia arquivo de imagem para Data Storage

```
az storage blob upload \
  --account-name "$DATA_STORAGE" \
  --container-name imagens \
  --name produto.jpg \
  --file ~/aie-cloud/aulas/04-servicos-cognitivos/lab/data/produto.jpg \
  --overwrite
```



Avaliação do arquivo Python

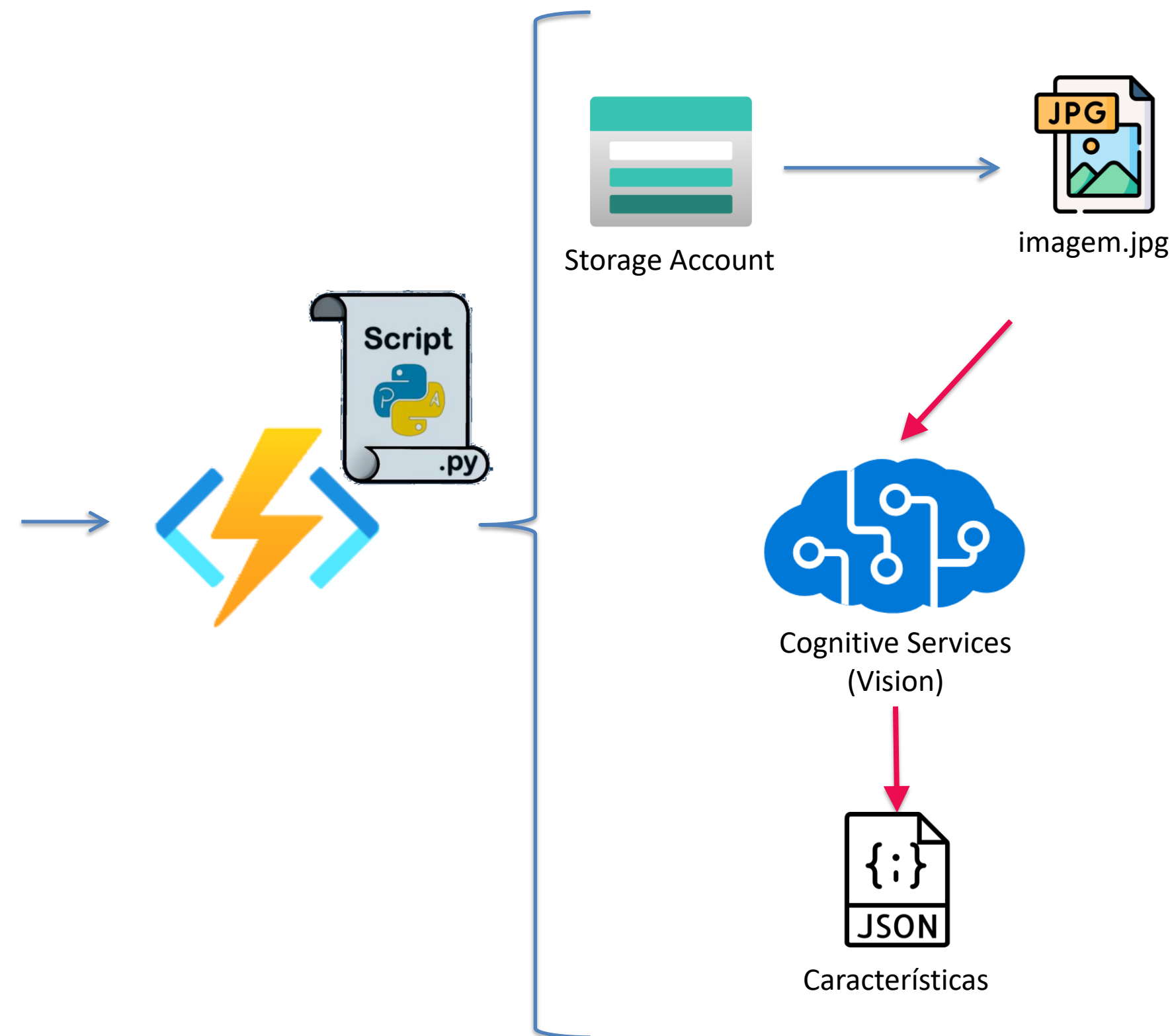
- Localize o diretório scripts usando o VS Code;
- Explore o arquivo function_app.py e a função analisar_imagem;
- Execute o Script Shell:

```
# Obtém Hostname da Function APP
export HOSTNAME=$(cd ~/aie-cloud/aulas/04-servicos-cognitivos/lab/terraform &&
terraform output -raw function_app_hostname)
echo "Host Name: $HOSTNAME"

# Exibe URL de Análise
echo "URL para analisar imagem: $HOSTNAME/api/analisa-
imagem?blob=produto.jpg"
```

Considere a "tool" e faça as discussões com a turma:

- Como trocar a imagem fixa por outra
- Tempo de Processamento
- Por que encapsular para o agente
- Troca de Provedor de solução
- Ex.: Pipeline aprovação de crédito imobiliário



EXERCÍCIO

Exercício Aula 4

Exercício que vale 10% da nota

Esta é a segunda entrega de grupo da disciplina. Ao todo são 5 entregas intermediárias (10% cada) + projeto integrado final entregue como ZIP 1 semana após a Aula 6 (50%, sem apresentação oral).

Como o grupo se organiza

Os 3 níveis de exercícios são divisão de trabalho dentro do grupo, não escolha individual livre:

- *Nível 1 — Básico: ecossistema cognitivo, pricing, segurança (API key vs MI), Vision capabilities*
 - *Nível 2 — Intermediário: pipeline robusto de reviews QC, casos de uso de Speech, pronto vs custom*
 - *Nível 3 — Avançado: bônus opcional — embeddings reais com Azure OpenAI (fecha o loop do AI Search da Aula 2), Custom Vision, sumarização de reviews via LLM*
- Mínimo obrigatório: N1 + N2 cobertos. N3 é bônus (até +2 pts extras na nota da aula).*

Repo Base: <https://github.com/elthonf/aie-cloud.git>

Item	Obrigatório?	Pontos máximos
Cabeçalho do grupo + distribuição do trabalho	✔ Sim	1 pt (Critério 4)
🟢 N1 — Exercícios 1.1 (pronto/custom/LLM), 1.2 (custo mensal), 1.3 (segurança MI vs API key), 1.4 (Vision capabilities)	✔ Sim	3 pts (Critério 1)
🟡 N2 — 2.1 (pipeline robusto: summary + PII + opinion mining), 2.2 (casos de Speech na QC), 2.3 (Vision pronto vs Custom Vision)	✔ Sim	3 pts (Critério 2) + 2 pts qualidade técnica (Critério 3)
🔴 N3 — 3.1 (embeddings reais com Azure OpenAI), 3.2 (Custom Vision), 3.3 (sumarização LLM)	🎁 Bônus	até +2 pts extras
Reflexão coletiva ao final	✔ Sim	1 pt (Critério 5)
Total		10 pts

Sugestão de nome de arquivo: `entrega-grupo-NN-aula04.zip` (substitua NN pelo nome de um integrante).

O que deve conter no arquivo

```
qc-grupo-NN-aula04/
├─ entrega-grupo-aula04.md    # ★ documento principal (template em ../template-entrega-grupo.md)
├─ terraform/                 # main.tf evoluído (AI Services+KV+Function + roles, e Azure OpenAI se N3.1)
├─ function/                  # function_app.py com rotas adicionais (sumarização se N2.1/N3.3)
├─ scripts/                   # Python: re-indexação com embeddings (se N3.1)
└─ diagramas/
    └─ arquitetura-qc-aula04.png # QC atualizada com a camada cognitiva (Speech/Language/Vision)
└─ README.md                  # Como rodar o N2/N3 (se incluído)
```

Destrua tudo. **Confirme o zero.**

✗ NÃO BASTA

Parar os recursos

Container parado, ACR ocioso — ainda há storage e registro reservados consumindo. Pausar não é zerar.

✓ FAÇA

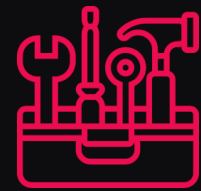
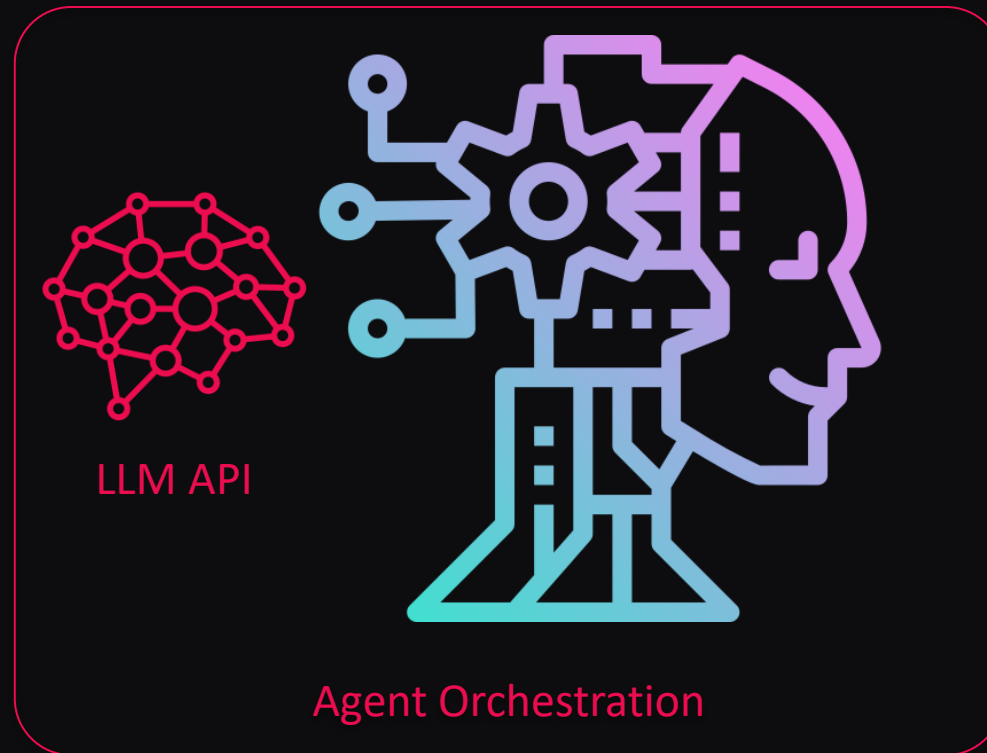
terraform destroy

Apaga RG, Function, Service Plan, ACR e ACI de uma vez.
Depois: Cost Management → confirme ~\$0.

```
terraform destroy -auto-approve
```

► PROJETO INTEGRADO · QUANTUM COMMERCE

a cinta de utilidades cognitiva



Custom Tools

Speaking Tools

Vision Tools

Listening Tools

Agent Tools

A3 **buscar no catálogo** — uma primeira tool simples

A4 **escutar** o cliente — Speech

A4 **entender** o sentimento — Language

A4 **ver** o produto — Vision

4 tools cognitivas no mesmo endpoint serverless

GET /produtos?categoria=... # (Aula 3)

POST /transcrever?blob=... # Speech

POST /analisar-reviews?limit=10 # Language

POST /analisar-imagem?blob=... # Vision

cada uma → uma tool via function calling,

e o LLM decide quando usar qual.

O que você leva desta aula

ECOSSISTEMA

Pronto · custom · LLM

Genérica e fechada → pronta.

Do seu domínio → custom.

Aberta e criativa → LLM.

SPEECH + SEGURANÇA

O agente escuta

A mesma vacina da Aula 3: Managed Identity em serviços cognitivos, sem API key no código.

LANGUAGE

O agente lê

Sentimento, entidades e PII pra LGPD — pipeline real de reviews no Document DB.

VISION + MATRIZ

O agente vê

Tags, OCR, objetos.

E a decisão pronto vs custom vs LLM — a entrega conceitual da aula.

A large blue circle is positioned in the top-left corner of the slide. A small, horizontal pink bar is located just below the bottom edge of this circle.

Obrigado.

Nos vemos na Aula 5